



Memoização no React: useMemo e React.memo

Aprenda como otimizar a performance das suas aplicações React evitando reprocessamentos desnecessários e melhorando significativamente o tempo de resposta dos seus componentes.

O que é memoização?

Memoização é uma técnica de **performance** que armazena o **resultado de uma função** para evitar que ela **rode novamente** se os dados **não mudaram**.

Funciona como um "**cache**" inteligente dentro do React, preservando valores calculados e evitando reprocessamento desnecessário de informações.



React Render times underabrevements

1000

12.25

2000

Render time

Answer time

Por que isso importa?



Re-renderizações frequentes

O React **re-renderiza** componentes com frequência, às vezes sem necessidade real.



Cálculos pesados repetitivos

Operações complexas podem ser executadas múltiplas vezes, consumindo recursos.



Melhoria de performance

Memoizar evita **retrabalho**, melhora **velocidade** e reduz **uso de recursos**.

Como o React ajuda com isso?

O React oferece ferramentas específicas para implementar memoização em diferentes níveis da sua aplicação:

Hook/Função	Para que serve?
useMemo	Memoriza o resultado de uma função
React.memo	Memoriza o componente inteiro



useMemo - o que faz?

```
const valorMemorizado = useMemo(() => {  
  return funcaoPesada();  
}, [dependencias]);
```

📌 **useMemo** só executa `funcaoPesada()` se as **dependências mudarem**.

✅ Evita executar **de novo** se nada mudou, armazenando o resultado anterior em memória.

Quando usar useMemo?



Filtrar listas grandes

Quando você precisa filtrar ou processar grandes conjuntos de dados que seriam custosos para reprocessar.



Cálculos complexos

Operações matemáticas pesadas que consomem CPU e poderiam causar lentidão na interface.



Transformações de dados

Manipulações de arrays/objetos que envolvem operações como map, filter, reduce ou sort.



Funções no JSX

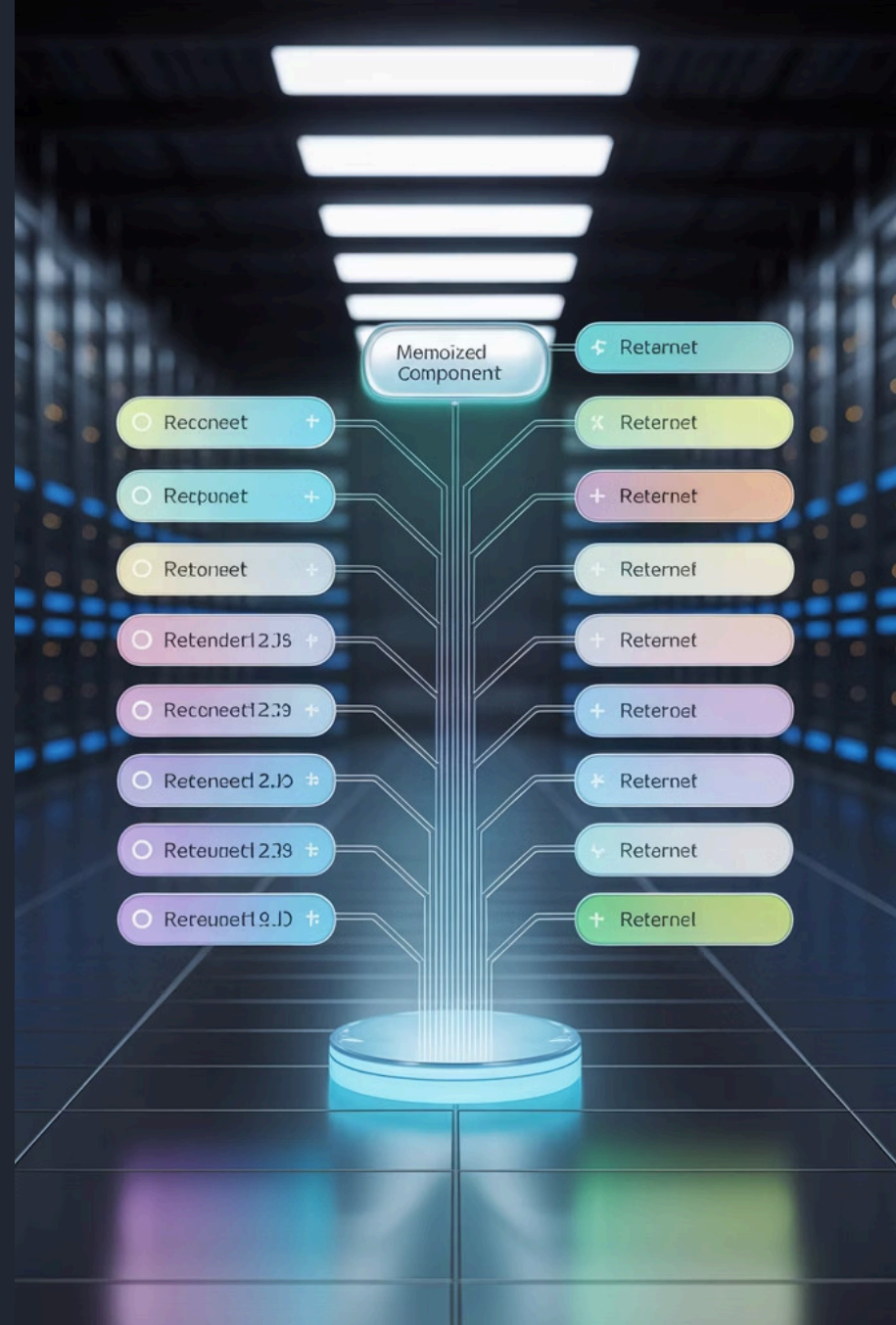
Funções que são chamadas diretamente dentro do JSX e poderiam ser recriadas a cada renderização.

React.memo - o que faz?

```
const MeuComponente = React.memo(function MeuComponente(props) {  
  return <div>{props.valor}</div>;  
});
```

📌 **React.memo** evita que o componente **seja renderizado novamente** se as props **não mudaram**.

É um HOC (High Order Component) que envolve o componente e faz uma comparação superficial das props.



Qual usar e quando?

$f(x)$

Problema: Cálculos repetitivos

Quando uma função pesada está sendo executada múltiplas vezes sem necessidade

Solução: `useMemo`

Memoriza apenas o resultado da função, mantendo o resto do componente normal



Problema: Re-renderizações em cascata

Quando componentes filhos estão sendo re-renderizados sem alteração nas props

Solução: `React.memo`

Memoriza o componente inteiro, evitando que ele seja renderizado novamente

Dica importante

⚠ Evite usar memoização em tudo.

Use **apenas** quando notar lentidão ou re-renderizações desnecessárias.

Por quê?

- A própria memoização tem um custo de memória
- Componentes simples geralmente não precisam
- Memoização excessiva pode complicar o código
- Pode até piorar a performance em alguns casos



O que vem agora?



Lista de produtos

Implementaremos um componente de catálogo com memoização para otimizar renderizações.



Filtro por categoria

Criaremos filtros de produtos usando useMemo para evitar reprocessamento a cada digitação.



Contador independente

Demonstraremos como isolar componentes com React.memo para evitar re-renderizações desnecessárias.

Na **próxima aula**, aplicaremos useMemo e React.memo em um **exemplo prático** completo!